Journal of Computational Physics 229 (2010) 8981–8993

Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp

A new adaptive mesh refinement data structure with an application to detonation

Hua Ji [a,*], Fue-Sang Lien [b], Eugene Yee [c]

[a] Waterloo CFD Engineering Consulting Inc., Waterloo, ON, Canada N2T 2N7
[b] Department of Mechanical Engineering, University of Waterloo, Waterloo, ON, Canada N2L 3G1
[c] Defence R&D Canada – Suffield, P.O. Box 4000, Medicine Hat, AB, Canada T1A 8K6

ARTICLE INFO

Article history:
Received 24 May 2010
Received in revised form 12 August 2010
Accepted 19 August 2010
Available online 25 August 2010

Keywords:
Adaptive mesh refinement/derefinement
Inviscid compressible flow
Detonation
Cell-based AMR data structure
Structured grid

ABSTRACT

A new **C**ell-based **S**tructured **A**daptive **M**esh **R**efinement (CSAMR) data structure is developed. In our CSAMR data structure, Cartesian-like indices are used to identify each cell. With these stored indices, the information on the parent, children and neighbors of a given cell can be accessed simply and efficiently. Owing to the usage of these indices, the computer memory required for storage of the proposed AMR data structure is only $\frac{5}{8}$ word per cell, in contrast to the conventional oct-tree [P. MacNeice, K.M. Olson, C. Mobary, R. deFainchtein, C. Packer, PARAMESH: a parallel adaptive mesh refinement community toolkit, Comput. Phys. Commun. 330 (2000) 126] and the fully threaded tree (FTT) [A.M. Khokhlov, Fully threaded tree algorithms for adaptive mesh fluid dynamics simulations, J. Comput. Phys. 143 (1998) 519] data structures which require, respectively, 19 and $2\frac{3}{8}$ words per cell for storage of the connectivity information. Because the connectivity information (e.g., parent, children and neighbors) of a cell in our proposed AMR data structure can be accessed using only the cell indices, a tree structure which was required in previous approaches for the organization of the AMR data is no longer needed for this new data structure. Instead, a much simpler hash table structure is used to maintain the AMR data, with the entry keys in the hash table obtained directly from the explicitly stored cell indices. The proposed AMR data structure simplifies the implementation and parallelization of an AMR code. Two three-dimensional test cases are used to illustrate and evaluate the computational performance of the new CSAMR data structure.

1. Introduction

Although an unstructured mesh is highly flexible for dealing with very complex geometries and boundaries [16], the discretization scheme used for these types of meshes (usually in conjunction with control-volume-based finite element methods) is generally much more cumbersome than those used for structured meshes. In particular, the data structure used by an unstructured mesh requires information on "forming points" that are needed to describe the (arbitrary) shape and location of the faces of control volumes where the conservation laws need to be satisfied and to define the adjacent neighbors for each of these control volumes through an appropriate connectivity matrix. These types of operations require generally indirect memory referencing and, as a consequence, efficient matrix solvers widely used for structured meshes cannot be easily adapted for unstructured meshes. Furthermore, memory requirements are generally higher when compared to the structured schemes. In view of this, computational methods based on unstructured meshes are generally computationally less

* Corresponding author. Tel.: +1 519 8856722.
E-mail address: huaji2005@gmail.com (H. Ji).

0021-9991/$ - see front matter © 2010 Elsevier Inc. All rights reserved.
doi:10.1016/j.jcp.2010.08.023

efficient than those based on structured meshes, particularly when compared to structured schemes that use a Cartesian grid. In order to utilize the computational efficiency afforded by a Cartesian structured grid while retaining the flexibility and accuracy in the discretization process required to address problems involving complex geometries and a wide range of spatial scales, adaptive mesh refinement (AMR) can be applied to focus the computational effort and memory usage where it is required for the accurate representation of the solution at minimal cost.

There are two main approaches that have been developed for structured grid AMR. The first approach utilizes a block-structured style of mesh refinement [5–7,20], in which a sequence of nested structured grids at different hierarchies or levels (cell sizes are the same for all grids at the same hierarchy or level) are overlapped with or patched onto each other. A tree-like (or, directed acyclic graph) data structure is usually used to facilitate the communication (transfer of information) between the regular Cartesian grids at the various hierarchies. It should be noted that each node in this tree-like data structure represents an entire grid rather than simply a cell and, as a result, efficient solvers for structured grids can be used to solve the system of algebraic equations resulting from the discretization of the governing equations on each node. Unfortunately, this type of AMR framework is limited in its flexibility in the sense that the clustering methods used to define the sub-grids in the hierarchy of Cartesian grids can result in portions of the computational domain covered by a highly refined mesh when it is not needed (e.g., solution is smooth in this region), resulting in a wasted computational effort.

In the second approach for AMR, the hierarchy of grids at different levels is represented by a quad-tree/oct-tree data structure in two/three dimensions (2-D/3-D), with each node representing an individual grid cell (rather than a block of grid cells as in the first approach). Because the mesh is only locally refined in the region where it is required, the second approach results in improved computational efficiency, increased storage savings, and better control of the grid resolution in comparison with the first approach. In a conventional oct-tree discretization and representation, such as that used by [18], the connectivity information between an individual cell and its neighbors needs to be stored explicitly, with each cell requiring 19 words of computer memory to maintain the oct-tree data structure. In addition to the large memory overhead required to maintain this oct-tree data structure, it is also very difficult to parallelize owing to the fact that the neighbors of each cell are connected (linked) each other. Consequently, removing or adding one cell in the process of adaptive mesh refinement will affect all the neighboring cells of this one cell. If the pointers to the neighboring cells are not stored explicitly, the oct-tree must be traversed to find these neighboring cells. On average, it is expected that two levels of the oct-tree must be traversed before a neighbor can be found, but in the worst-case scenario it may be necessary to traverse the entire tree to access the nearest neighbors. Tree traversals of this type can extend from one processor to another, making it difficult to vectorize and parallelize these operations.

The fully threaded tree (FTT) [12] has been developed to reduce the memory overhead required to maintain the information embodied in a tree structure and to facilitate rapid and easy access to the information stored in the tree. In the FTT structure, all cells are organized in groups referred to as octs. Each oct maintains pointers to eight (child) cells and to a parent cell. Each oct stores information about its level (equal to the level of the parent cell) and maintains the information about its position in the computational domain. Furthermore, each oct maintains pointers to the parent cells of the six neighboring octs, and each cell of the oct maintains a pointer to its child oct (or to a null pointer if the cell has no children).

The memory requirements for the FTT structure is 19 words per oct or $2\frac{3}{8}$ words per cell. The memory overhead is significantly reduced compared to that used in the conventional oct-tree structure. Another advantage of the FTT structure is that the actual number of traversed levels required to find a neighbor never exceeds one. However, the FTT is still based on a tree data structure. In a tree-based code for AMR, a complete tree traversal starting from the coarsest cell (i.e., root cell) is frequently required to access a given cell in the adaptive mesh. To avoid a complete traversal of the tree from the root cell, a linked list is frequently used to improve the access efficiency in the tree. However, the use of a linked list incurs a computer memory overhead. Furthermore, it may be very difficult to access an arbitrary cell in the tree using a linked list in a parallel code, owing to the fact that the parent of a particular cell may not even reside on the same processor.

In this paper, we propose a new and more transparent AMR data structure, which we refer to henceforth as the **C**ell-based **S**tructured **A**daptive **M**esh **R**efinement (or CSAMR) data structure. In the CSAMR data structure, Cartesian-like indices are used to identify each cell. With these stored indices, the information on the parent, children and neighbors of a given cell can be accessed simply and efficiently. Owing to the usage of these indices, the computer memory required for storage of the proposed AMR data structure is only $\frac{5}{8}$ word per cell, in contrast to the conventional oct-tree [18] and FTT [12] data structures which require, respectively, 19 and $2\frac{3}{8}$ words per cell for storage of the connectivity information. Because the connectivity information (e.g., parent, children and neighbors) of a cell in our proposed AMR data structure can be accessed using only the cell indices, a tree structure which was required in previous approaches for the organization of the AMR data is no longer needed for this new data structure. Instead, a simpler hash table structure can be used to maintain the AMR data, and the determination of the entry keys in this hash table is obtained straightforwardly from the explicitly stored indices. Using our proposed data structure, an AMR code can be more easily parallelized than one based on a tree structure because one is no longer concerned as to whether the parent of a cell resides on the same processor.

The paper is organized as follows. In Section 2, the new CSAMR data structure and its implementation are described in detail. Section 3 briefly summarizes the governing equations for the test cases and the numerical method used to solve these equations. Results from two three-dimensional test simulations (namely, the spherical Sedov blast wave test case and a 3-D detonation case) are presented in Section 4. The computational performance of our new CSAMR data structure is evaluated and compared to that based on the FTT data structure [12]. Our conclusions are summarized in Section 5.

2. Proposed AMR data structure and its implementation

In our proposed AMR data structure, all cells are organized in oct-based groups similar to the FTT structure. However, in our proposed data structure, each oct (see Fig. 1) only stores an oct level, an oct flag that indicates whether the oct is further refined, and oct indices (i.e., i, j, and k similar to Cartesian indices) which embody all the information required to access and maintain the AMR data structure. The computer memory overhead required to store our proposed AMR data structure is only $\frac{5}{8}$ word per cell, which is significantly less than the 19 words per cell and $2\frac{3}{8}$ words per cell required for the conventional oct-tree and FTT structures, respectively.

2.1. Construction of AMR data structure

In this subsection, we use a two-dimensional domain as an example to show how to construct an adaptive mesh with our proposed AMR data structure. To this purpose, consider the rectangular domain shown in Fig. 2(a) with dimensions L_x by L_y, and with the southwest corner at (x_0, y_0). The domain is initially covered by a single oct, which we identify using the standard structured grid indexing as oct $(i,j) = (0,0)$. The oct is also identified by a second integer (level) that defines its level of refinement. Initially, the oct level is zero, representing the coarsest possible level as shown in Fig. 2(a). Fig. 2(b) shows the domain after one refinement. In this case, the refinement level of the four octs in now one (viz., level = 1). The (i,j) indices of each oct stores the standard structured indices as if the entire grid were at the refinement level of the oct. Fig. 2(c) shows the mesh and associated indices after one further refinement has been performed on the northeast oct in Fig. 2(b).

Using the stored index information, the centroid (x_c, y_c) and dimensions $(\Delta x, \Delta y)$ of each oct can be calculated explicitly as follows (where level below denotes the oct level):

$$(x_c, y_c) = \left(x_0 + (i + 1/2)\frac{L_x}{2^{\text{level}}}, y_0 + (j + 1/2)\frac{L_y}{2^{\text{level}}}\right), \tag{1}$$

$$(\Delta x, \Delta y) = \left(\frac{L_x}{2^{\text{level}}}, \frac{L_y}{2^{\text{level}}}\right). \tag{2}$$

The indices of the four children octs (i_s, j_s) can be easily calculated using the stored index information as

$$(i_s, j_s) = \{(2i + m, 2j + n) | m = 0, 1; n = 0, 1\}. \tag{3}$$

Finally, the indices of the parent oct (i_p, j_p) can be calculated as

$$(i_p, j_p) = \left(\text{int}\left[\frac{i}{2}\right], \text{int}\left[\frac{j}{2}\right]\right), \tag{4}$$

where int $[\cdot]$ denotes the integer value of the quantity contained in the following square brackets. Because computing integer powers of two and adding or subtracting an integer value to or from a corresponding index in the preceding equations can be

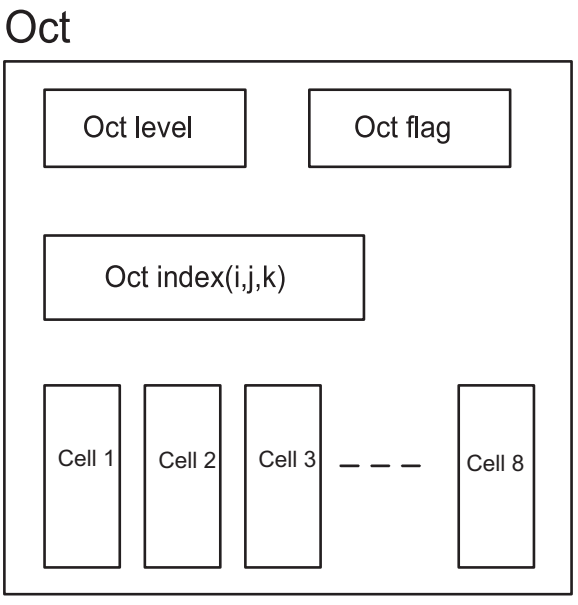

Fig. 1. Proposed AMR data structure for an oct.

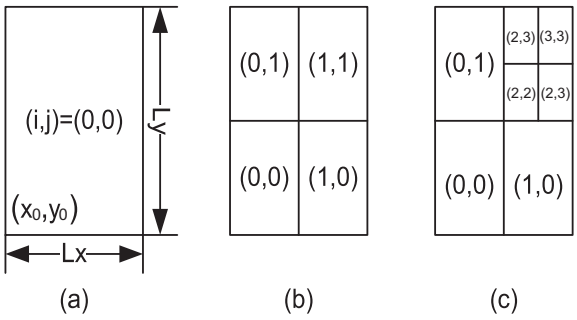

Fig. 2. Oct indices (i,j) for (a) an initial mesh consisting of one oct, (b) a mesh after one level of refinement, and (c) a mesh after two levels of refinement.

implemented very efficiently, each oct's geometric data and the information concerning the parent and children indices can be calculated on an as-needed basis, resulting in a reduction in the computer memory required for storage of the connectivity information as compared to the conventional tree and FTT data structures. As one can imagine, accessing the connectivity information of a particular cell in the adaptive mesh is greatly simplified by this feature.

As mentioned earlier, a linked list is frequently used in conjunction with a tree-based representation for AMR in order to reduce the frequency for which a complete tree traversal (starting from the root cell) is required in order to access a given cell in the tree. Unfortunately, the use of a linked list has the disadvantage in that it results in an additional overhead in computer memory required to store the connectivity information. Furthermore, the use of a linked list complicates the parallel implementation of AMR vis-à-vis the access to an arbitrary cell in the tree (e.g., the parent of a given cell may not even reside on the same processor). Our proposed AMR data structure overcomes these deficiencies by using a hash table technique to store and search over the large amount of data embodied in the AMR. The computational effort to access a random datum in a hash table structure is $O(1)$, compared to $O(\log N)$ (N is the total number of cells) in a binary tree structure. Furthermore, a hash table is easier to implement in a parallel code because we do not have to concern ourselves as to whether the parent of a cell resides on the same processor.

In our proposed AMR data structure, we use the open source library glib (http://www.gtk.org/) in our implementation of the hash table technique. This library provides functions to insert, remove and retrieve data in the hash table. To use the hash table structure, we simply need to provide a unique key in order to identify each oct in the hash table. To this purpose, the explicitly stored Cartesian-like oct indices provide one possible method to encode these entry keys in the hash table. More specifically, using the readily available oct indices, the unique entry key (ID) for an oct can be calculated simply as follows:

$$\mathrm{ID} = \sum_{l=0}^{\mathrm{level}-1} (2^l \cdot 2^l) + i \cdot 2^{\mathrm{level}} + j, \tag{5}$$

where (i,j) is the oct index, and level denotes the oct level.

In a numerical simulation of fluid flow, it is often necessary to use a flow variable value in a neighboring cell in order to calculate the gradient of the variable. Using the stored oct indices and the hash table, the neighboring cells in our data structure can be accessed conveniently without the need to store the connectivity information or to traverse a tree structure in order to locate all the neighbors. As such, our proposed data structure not only reduces the computer memory required to store the connectivity information, but also facilitates the parallel implementation of AMR because the connectivity information no longer needs to be updated after either a refinement or derefinement of the mesh. In the FTT structure, no neighboring cells with a level difference greater than one are permitted in order for the FTT data structure to be maintained during the refinement/derefinement process of AMR. Although this constraint can be removed completely using our proposed data structure, we will nevertheless retain this constraint in order to simplify the implementation.

To be more specific on how one would use our proposed AMR data structure to locate the neighbors of a cell, let us consider an oct with index (i,j) [see Fig. 3(a)]. Cell 3 in Fig. 3(a) is one of the four cells contained in the oct (i,j). We would like to determine the neighboring cells of Cell 3. The western and southern neighbors can be obtained directly because they are located in the same oct as Cell 3. Fig. 3 illustrates the three possibilities for the eastern neighbor(s) of Cell 3. To find the eastern neighbor of Cell 3, we first look for the eastern oct neighbor $(i+1,j)$ of oct (i,j) using the hash table, with the entry key calculated in accordance to Eq. (5). If oct $(i+1,j)$ exists in the hash table, then Cell 2 of oct $(i+1,j)$ [see Fig. 3(a)] must be the eastern neighbor residing at the same level as Cell 3. However, if Cell 2 is not a leaf cell, then two eastern neighbors residing at the next higher level must be present in the oct $(2i+2,2j+1)$ [see Fig. 3(b)]. Moreover, if oct $(i+1,j)$ does not exist in the hash table, this indicates that the eastern neighbor of Cell 3 must be at the next coarser level of refinement. The eastern neighbor at this coarser level must then be located in the oct $(\mathrm{int}[(i+1)/2], \mathrm{int}[j/2])$ [see Fig. 3(c)] because of our imposed constraint that no neighboring cells with a level difference greater than one are allowed in the mesh. Therefore, at most, two searches are sufficient to find a neighbor of a given cell. However, it should be noted that there are half neighbors which can be obtained directly without using the hash table. Statistically then, the average number of searches required to find a neighbor of a given cell is one.

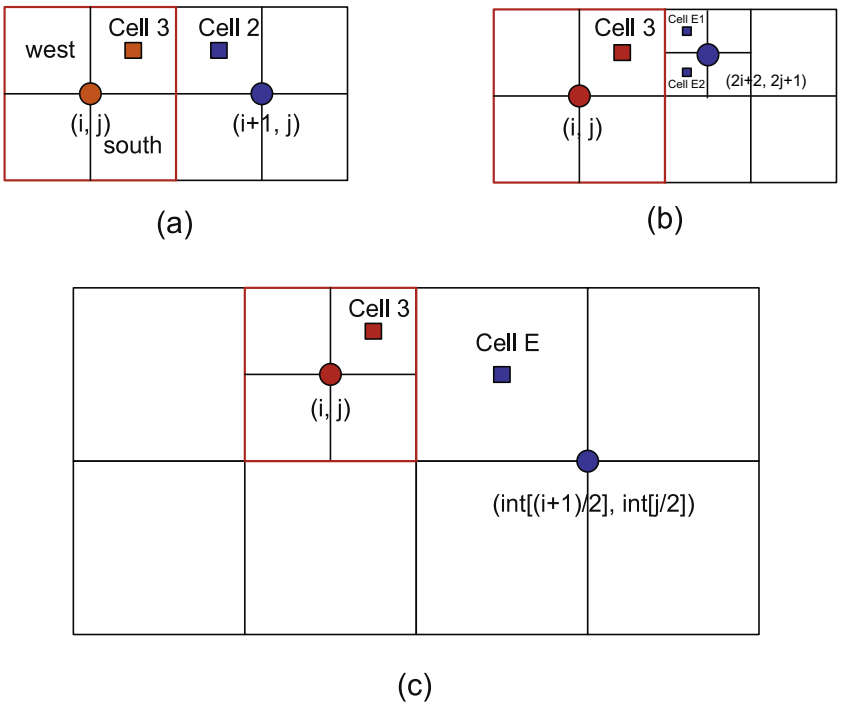

Fig. 3. Determination of the eastern neighbor(s) of Cell 3: (a) neighbor cell at the same level; (b) neighbor cells at the next higher refinement level at a coarse-fine boundary transition; and, (c) neighbor cell at next lower refinement level in a fine-coarse boundary transition.

3. Numerical methods

3.1. Governing equations

The numerical methods used in this paper have been fully described in [11] and, as a consequence, only the relevant details of the numerical methods required for interpretation of the numerical simulations (described later in this paper) will be presented here. The Euler equations used for the numerical simulations assume the following form:

$$\begin{aligned}
&\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x_j}(\rho u_j) = 0,\\
&\frac{\partial (\rho u_i)}{\partial t} + \frac{\partial}{\partial x_j}(\rho u_i u_j + p\delta_{ij}) = 0,\\
&\frac{\partial (\rho E)}{\partial t} + \frac{\partial}{\partial x_j}(u_j(\rho E + p)) = 0,
\end{aligned} \tag{6}$$

where ρ denotes the density, u_i denotes the component of the velocity in the x_i direction, p is pressure, E is the total energy and δ_{ij} denotes the Kronecker symbol ($\delta_{ij} = 1$ if $i = j$, and $\delta_{ij} = 0$ otherwise). The total energy is related to the internal energy as $E = e + (u_i u_i)/2$, where e is the internal energy.

In addition to Eq. (6), for physical systems involving chemical reactions (such as for detonation) a reaction model is also needed. To this purpose, we consider an ideal gas and a single-step irreversible reaction model. Assume that the ideal gas mixture consists of two components (A, B) having the same molecular weight which are related through the single irreversible reaction $A \rightarrow B$. If we assume that the heat capacity of these components have the same value and are constant, the equation of state for the mixture assumes the following form:

$$\begin{aligned}
&p = \rho R T,\\
&e = \frac{RT}{\gamma - 1} + (1 - Z)q,
\end{aligned} \tag{7}$$

where Z denotes the mass fraction of the product B, q is the heat of reaction, R is the universal gas constant, T is the temperature and γ is the ratio of specific heats at constant pressure and volume. The reaction rate is given by the Arrhenius rate law, so

$$\frac{\partial (\rho Z)}{\partial t} + \frac{\partial}{\partial x_j}(\rho u_j Z) = \kappa \rho (1 - Z) \exp\left(-\frac{E_A}{RT}\right), \tag{8}$$

where κ is the pre-exponential factor (frequency factor) and E_A is the activation energy.

An operator splitting technique (or, the method of fractional steps) for the computation of the time-dependent reactive flow (see [19]) is used to solve Eq. (8). This technique allows a decoupled treatment for the time-implicit discretization of the source term and the time-explicit discretization of the hydrodynamic transport term. The inviscid flux in Eq. (6) is calculated by solving a local Riemann problem at each control surface using the Harten, Lax and van Leer for contact wave (HLLC) approximate Riemann solver [4,23]. The semi-discrete form of Eq. (6) is advanced in time using a three-stage explicit Runge–Kutta scheme [3].

The reconstruction procedure used for evaluation of the fluxes through the bounding faces of a control volume and the refinement criteria used for the adaptive mesh refinement are described in [11]. It should be noted that for a tree-based data structure (such as the FTT structure), creating or destroying a child oct is a computationally expensive process because it involves a reorganization of the tree structure. In striking contrast, for our proposed data structure, there is no need to explicitly update the information concerning the parent, children and neighbors of an oct in the creation or destruction of a child oct. Rather, all that is necessary is to simply insert or delete an oct in or from the hash table when a child oct is either created or destroyed, respectively.

3.2. Grid partitioning

To address the grid partitioning and associated mapping problem in this paper, the Hilbert space-filling curve (SFC) approach is used. There are three important properties associated with this partitioning approach: namely, mapping, locality and compactness [1]. Fig. 4 shows an example of a Hilbert (or U-ordering) space-filling curve which can be exploited as a grid-partitioner in parallel computations [8,13,14]. This involves mapping the points in a higher-dimensional space to the corresponding points on a space-filling curve. Such a curve can be generated recursively. To use this curve to decompose a higher-dimensional space into various domains and to map these domains to the available processors so that each processor is subject to an equal workload, we only need divide the points along the curve into equal-length segments and map the domains in the higher-dimensional space corresponding to the points in these segments to the available processors (one processor for each domain in the higher-dimensional space or, equivalently, for each segment of the associated Hilbert SFC).

A key property of the Hilbert space-filling curve is its compactness. This property implies that only the coordinates of a point in the higher-dimensional space are required for the determination of the corresponding mapped location on the one-dimensional space-filling curve (line). This property is important because the space-filling curve can be constructed in parallel without the need to exchange any information between the various processors. It is noted that this property is unique to the Hilbert SFC (viz., other SFC partitioners such as the Morton SFC do not have this property). More specifically, for the Hilbert SFC, the adjacent neighboring points along the one-dimensional curve correspond to the face-neighboring cells in the higher-dimensional computational space (see Fig. 4), hence ensuring the locality property of this SFC. Owing to this locality, all face-neighboring cells in a domain are mapped to the same processor, which has the concomitant advantage that it reduces the computational overhead incurred in the interprocessor communications.

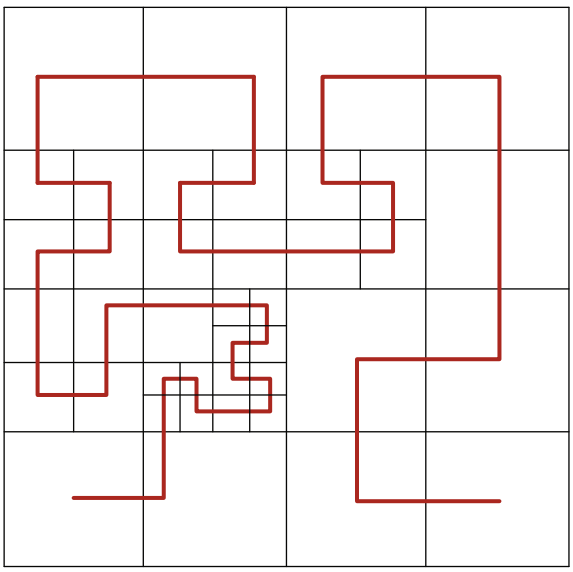

Fig. 4. The mapping of points in a computational domain to a Hilbert space-filling curve for use in grid partitioning in a parallel computation.

4. Numerical results

Two numerical examples utilizing our proposed AMR data structure are described in this section. These two examples correspond to three-dimensional test cases which apply the proposed data structure within an implementation of an AMR flow solver, the latter of which is executed on a distributed memory parallel computer using domain decomposition.

4.1. Spherical Sedov blast wave test

We consider a three-dimensional Sedov blast wave test case. In three dimensions, the Sedov blast wave propagates outwards spherically. The computational domain is a cube with unit side length. Within this cube, an ideal gas with $\gamma = 1.4$, initial density $\rho_0 = 1$, and initial pressure $p_0 = 10^{-5}$ is assumed to be at rest at the initial time $t = 0$. To initiate the explosion, a total energy $E = 1$ is deposited in a single cell at the southwest bottom corner of the computational domain. The simulation was undertaken using 5 levels of refinement from level = 5 to 9, inclusive. An equivalent simulation using a uniform grid would have required $512 \times 512 \times 512$ (or, about 134 million) cells. Fig. 5 displays the adaptive mesh at $t = 0.1$. The total number of cells in this mesh is 807,680. Fig. 6 compares the predicted results for the density and pressure obtained from the numerical simulation with the corresponding results for the density and pressure obtained from the analytical solution [21] at time $t = 0.1$. It is seen that the numerical solutions for the density and pressure are in excellent agreement with the analytical results.

4.2. Three-dimensional detonation front

Although it is known experimentally that detonations exhibit an inherently three-dimensional structure, most simulations conducted to date have considered only the two-dimensional case. While the computational expense of a two-dimensional detonation front simulation having a resolution necessary to resolve the relevant physics is relatively modest, the computational effort required for a full three-dimensional computation is both large and prohibitive. Deiterding [9] used a block-structured AMR with a full second-order accurate multi-dimensional discretization scheme to simulate a three-dimensional detonation case. Following his example, we consider the simulation of an unstable three-dimensional detonation front with parameters $\gamma = 1.2$, $E_A = 10$, $q = 50$, and overdriven factor $f \equiv D/D_{CJ} = 1.2$ (where D is the detonation speed and D_{CJ} is the detonation speed at the Chapman–Jouguet (CJ) point). The von Neumann point is placed at $x = 12$ on a $[0,20] \times [0,5] \times [0,5]$ computational domain. The transverse wave is initiated by increasing the pressure by 15% in the two pockets $[11.45,11.95] \times [0.0,0.25] \times [0.0,5.0]$ and $[11.45,11.95] \times [0.0,5.0] \times [1.45,1.95]$. The adaptive grid has $128 \times 32 \times 32$ cells at the coarsest level and two additional levels of refinement.

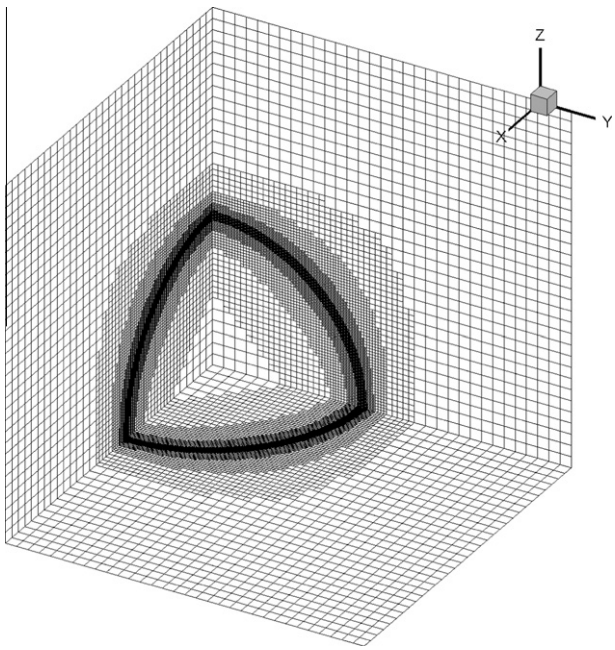

Fig. 5. Adaptive mesh at time $t = 0.1$ for the spherical Sedov blast wave test case.

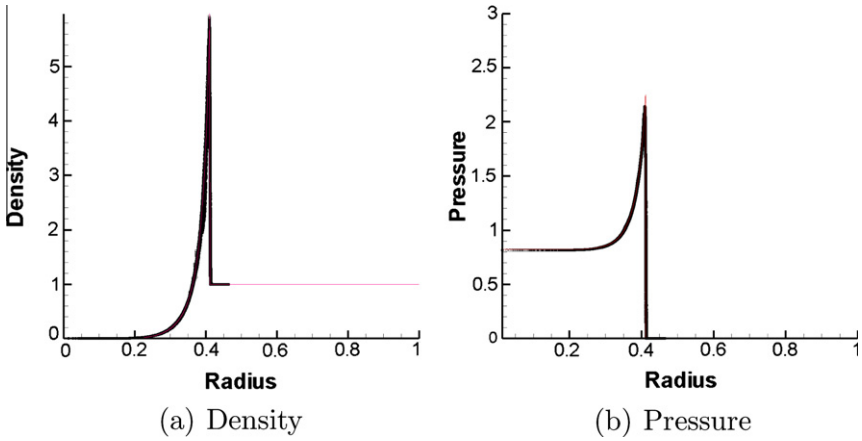

Fig. 6. Spherical Sedov blast wave test: (a) density and (b) pressure as a function of radius for time $t = 0.1$. The prediction (open circles) of these quantities is compared to the corresponding analytical solution (solid line).

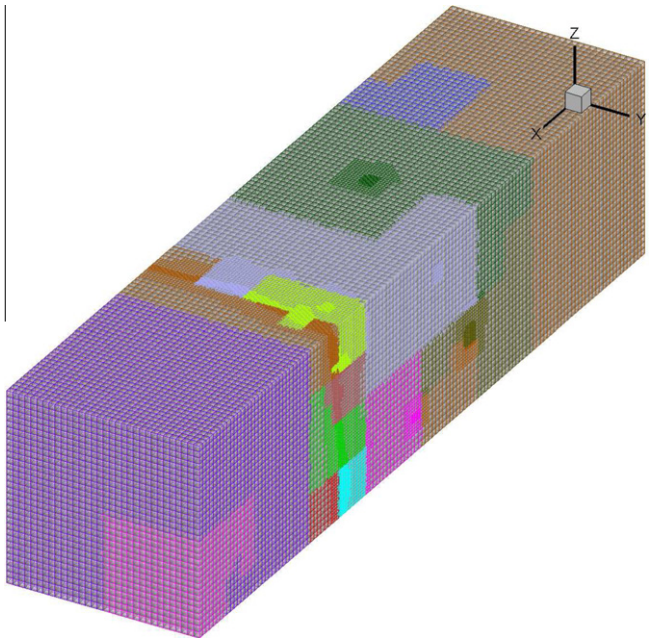

Fig. 7. The adaptive mesh for the three-dimensional detonation case partitioned on 16 processors.

Fig. 7 shows a snapshot of the adaptive three-dimensional mesh as partitioned on 16 processors. The simulations were undertaken on the SAW computer cluster, which forms part of the grid of high-computational performance clusters that comprise the Shared Hierarchical Academic Research Computing Network (SHARCNET) [22]. The SAW computer cluster consists of a group of 2,704 64 bit high-performance Xeon processors (2.83 GHz clock rate), each of which has 16 GB of local random access memory (RAM), that are linked together with a high-speed InfiniBand interconnection. The wall clock time to execute the simulation in one detonation cell period (about 3.2) is roughly 1.5 h. As a comparison, the computational time cited in [9] to conduct the same simulation was about 2 days using 32 processors (Athlon processors with a clock speed of 1.4 GHz).

For the three-dimensional detonation case, the average number of cells used for the simulation is about 500,000. With 16 processors, the wall clock time required to execute one time step is about 1.38 s. Using the Hilbert space-filling approach for grid partitioning, a speedup of about 15 (implying a parallel efficiency of about 92%) was achieved with 16 processors, compared to that obtained with a single processor.

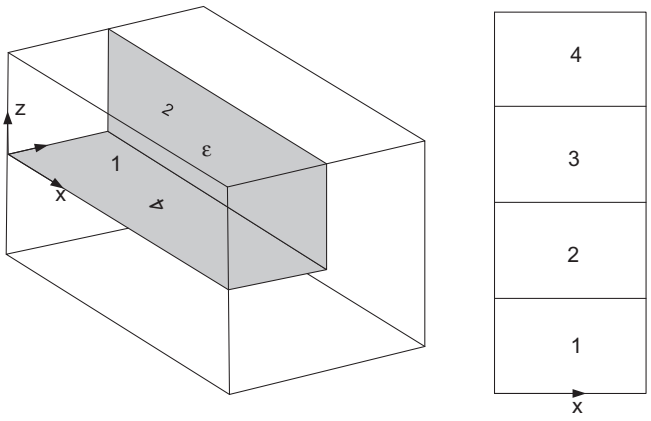

Fig. 8. Location and orientation of the two-dimensional roll-ups.

Figs. 9 and 10 show the periodic change of the three-dimensional detonation structures during one detonation cell. The computational domain has been mirrored twice. The 3-D graphical depiction here shows an isosurface of the detonation front. The 2-D graphics depiction exhibits roll-ups of the four sides of the cube marked in the 3-D plots (see also the sketch in Fig. 8). The figures display the numerical Schlieren plots of ρ, which is defined as [2]

$$v = 0.8 \exp\left(-\mu \frac{|\nabla \rho|}{|\nabla \rho|_{\max}}\right), \tag{9}$$

where μ is a positive amplification parameter.

Our results are in agreement with the block-structured AMR results in [9]. The isosurfaces of the detonation front clearly represent the four triple point lines formed by the transverse waves. During a complete detonation cell, the four lines remain mostly parallel to the boundary and hardly disturb each other. Each triple point line is driven forward by a Mach stem line into an incident shock region. A region which is a Mach stem in one direction can be either a Mach stem or an incident shock in the orthogonal direction. The same is true for the incident shock and we consequently have three different types of rectangular shock regions at the detonation front: Mach stem–Mach stem (MM), Incident–Incident (II) and mixed type regions (MI) (see also the sketch in Fig. 11) [24,9]. An MM region is bounded only by Mach stems and expands in the y- and z-directions. An II region is formed only by incident shocks and shrinks in both directions. The MI region is of mixed type, expanding in one direction while shrinking in the other.

4.3. Algorithm performance

Because the information on the parent, children and neighbors of a cell are not stored explicitly in our proposed AMR data structure, the code needed to maintain the tree structure (e.g., FTT structure [12]) is no longer required. As a consequence, an AMR code using our proposed data structure consists of three parts: (1) the fluid dynamics subroutines; (2) the refinement and derefinement subroutines; and, (3) the services subroutines required to initiate the code and output the numerical results of the simulation either for restarting or plotting.

Table 1 summarizes the number of lines of code used for each of these three functions, as well as the number of lines of code for an implementation that utilizes the FTT structure [12]. In the latter case, additional lines of code are required to maintain the tree structure. From Table 1, it is seen that the number of lines of code required in our implementation of (1) the fluid dynamics, (2) the refine and derefine operations and (3) the services parts of the program is comparable to that used in [12] (i.e., 2424 vs. 2712 lines of code,[1]). However, it should be noted that in contrast to the code used in [12], our code for this part of the implementation includes also several subroutines that are specific for detonation problems [e.g., code implementing numerical methods needed for the solution of Eq. (8)]. The number of lines of code required for the other two parts of the implementation are fewer than the corresponding parts of the implementation in [12].

The FTT portion of the code in [12] (which involves modifications of the tree structure to update the connectivity information in the AMR data and finding the neighbors, parent and children of a cell in the data structure) utilizes about 20% of the total computational time in a typical adaptive mesh fluid dynamics simulation. In contrast, in our CSAMR code which is based on a hash table (rather than a tree) structure, the computational time required for modification of the hash table (following creation or destruction of an oct) and for finding the neighbors, parent and children of a cell uses about 10% of the total computational time in a typical adaptive mesh fluid dynamics simulation. For example, in the simulation of the

[1] Note that the number of lines of code for FTT from [12] in Table 1 and the number of lines of code associated with the hash table from the open source library glib required in our CSAMR data structure are not included in the code line count reported here.

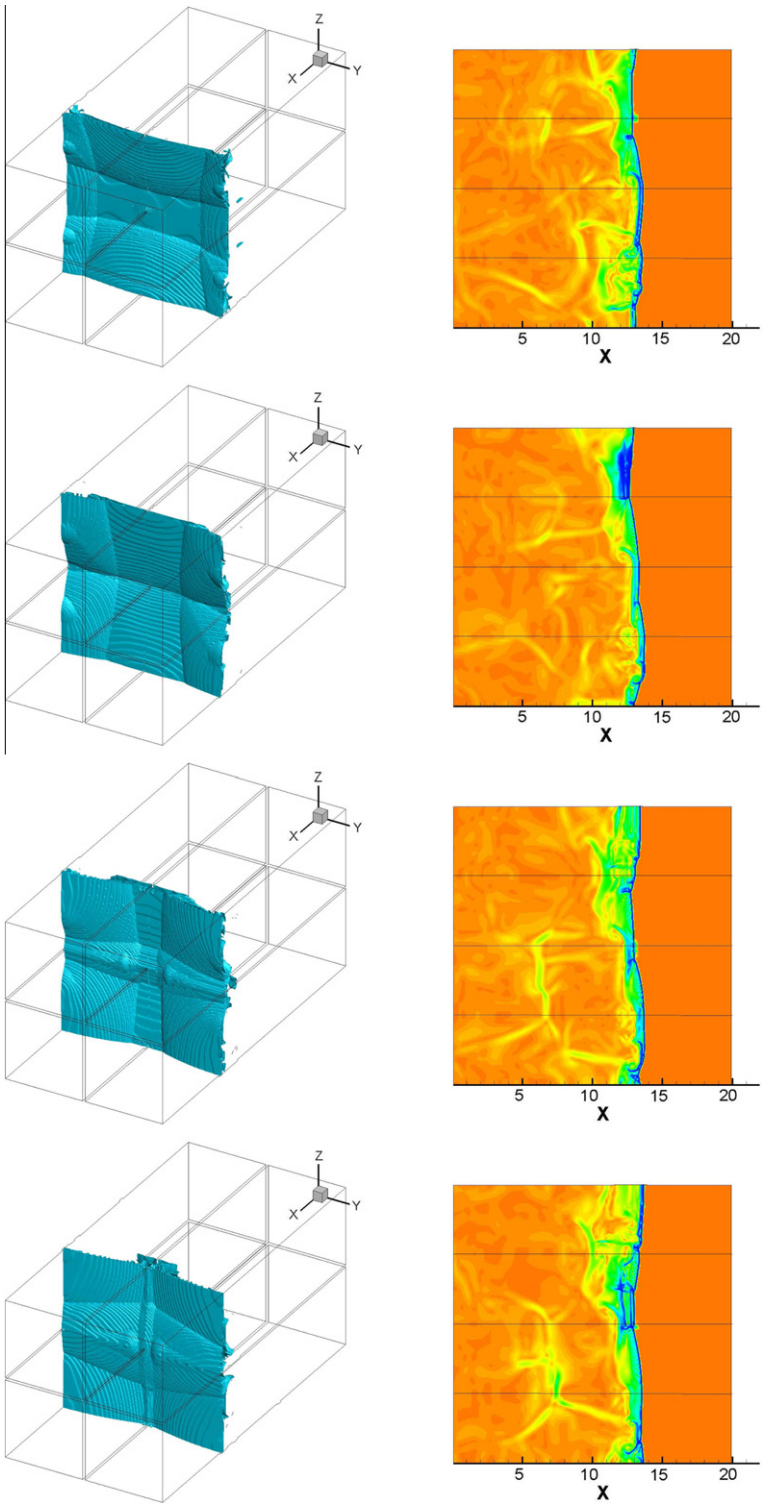

Fig. 9. Three-dimensional isosurfaces of the detonation front (left) and 2-D roll-ups visualized in the numerical Schlieren plots of density (right) for the three-dimensional detonation front at times t = 35.6, 36.0, 36.4 and 36.8 (from top to bottom).

three-dimensional detonation front described in Subsection 4.2, the computational time involved in maintenance of the hash table and in finding the neighbors, parent and children of a cell required only 9.6% of the total computational time for the

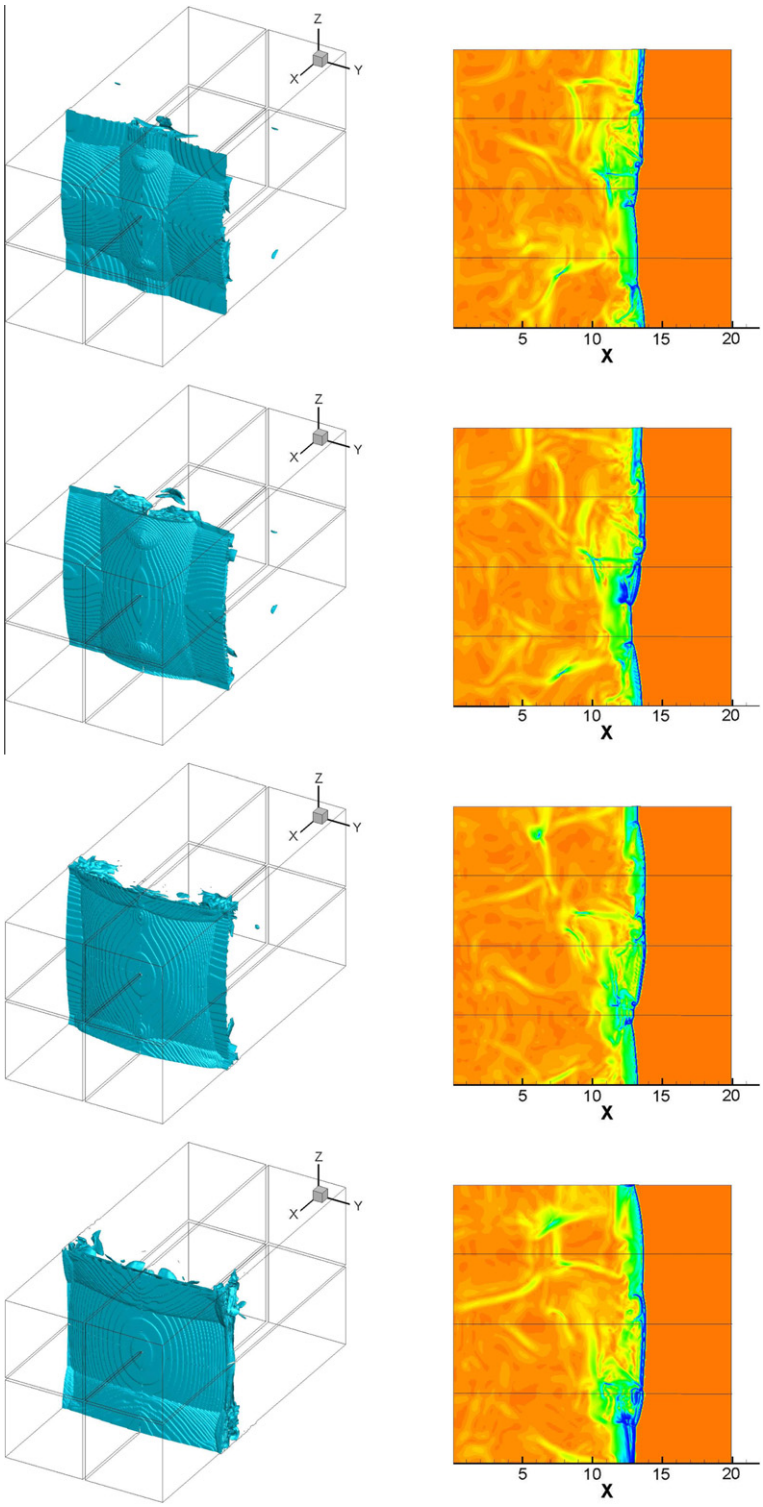

Fig. 10. Three-dimensional isosurfaces of the detonation front (left) and 2-D roll-ups visualized in the numerical Schlieren plots of density (right) for the three-dimensional detonation front at times $t = 37.2$, 37.6, 38.0 and 38.4 (from top to bottom).

fluid dynamics simulation. These performance tests suggest that our proposed hash-table-based CSAMR data structure is about twice as efficient as that which uses the tree-based FTT data structure for maintenance of the connectivity information

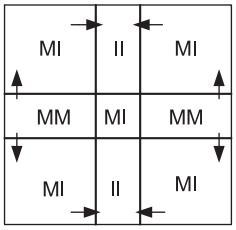

Fig. 11. Sketch of the periodic rectangular shock structure for $t = 36.4$.

Table 1
Number of lines of code required for the current implementation of AMR compared to implementation based on FTT [12].

Function	Number of code lines (current)	Number of code lines [12]
Fluid dynamics	1594	1485
Refine and derefine	563	601
Services	267	626
FTT		800

in the AMR data (following creation or annihilation of an oct) and for neighbor searching and accessing a cell in the data structure.

5. Conclusions

A new **C**ell-based **S**tructured **A**daptive **M**esh **R**efinement data structure, referred to herein as CSAMR, has been developed. In our proposed AMR data structure, Cartesian-like indices are used to identify each cell. Owing to the use of these indices, the computer memory required to store information concerning the connectivity of the mesh is significantly reduced in comparison to AMR data structures based on an oct-tree or on FTT. In particular, our proposed AMR data structure requires only the storage of $\frac{5}{8}$ word per cell, rather than 19 words per cell and $2\frac{3}{8}$ words per cell needed, respectively, for the conventional oct-tree and FTT data structures. Because the connectivity information (e.g., parent, children and neighbors) associated with a cell can be obtained from the indices, a complex tree structure is no longer required to maintain the AMR data. Instead, a simpler hash table structure can be used to organize and store the AMR data. The explicitly stored indices can also be used to calculate the entry keys for the octs in the hash table. The proposed AMR data structure greatly simplifies the parallelization of the code with very good load-balancing, in comparison to an implementation based on a tree structure. Two three-dimensional test cases involving detonation problems have been used to test the implementation of the new AMR data structure. Using these examples, we have demonstrated that the computational overhead incurred in using the CSAMR data structure (based on a hash table) for maintenance of connectivity information of the AMR data (following creation or annihilation of an oct) and for accessing a cell or neighbor searching in the data structure utilizes only about 10% of the total computational time in a fluid dynamics simulation. In contrast, the tree-based FTT data structure in [12] uses about 20% of the total computational time in a fluid dynamics simulation on these types of operations.

Similar to other Cartesian grid methods, either the Immersed Boundary Method or the cut-cell approach can be combined with the present AMR data structure to simulate flows over (moving) obstacles with arbitrary shapes (see, [10],[25],[11]). Although only an explicit time-stepping density-based compressible flow solver is considered in the present study, other popular pressure-based algorithms such as the (semi-implicit) fractional-step method [15] and the SIMPLE pressure-correction scheme [17] can also benefit from the utilization of the CSAMR data structure. Furthermore, a (geometric) multigrid solver can be easily incorporated to solve with high efficiency the system of equations arising from the discretization of the problem (e.g., [10]). The only limitation resulting from the use of Cartesian-like indices in the CSAMR data structure is that this structure cannot be applied to unstructured-grid methods. Generally speaking, numerical discretization on a structured grid (either in a Cartesian or curvilinear coordinate system) is more straightforward than on an unstructured grid. Therefore, the afore-mentioned limitation should not be considered as a major deficiency of the CSAMR data structure.

Acknowledgments

The computations conducted in this study were made possible by the facilities of the Shared Hierarchical Academic Research Computing Network (SHARCNET).

References

[1] M.J. Aftosmis, M.J. Berger, G. Adomavicius, A parallel Cartesian approach for external aerodynamics of vehicles with complex geometry, in: Proc. Therm. Fluids Anal. Workshop 1999, NASA Marshall Spaceflight Center, Huntsville, AL, 1999.
[2] M. Arienti, A Numerical and Analytical Study of Detonation Diffraction, Ph.D. thesis, California Institute of Technology, 2003.
[3] T.J. Barth, D.C. Jespersen, The design and application of upwind schemes on unstructured meshes, AIAA Paper 89-0366, 1989.
[4] P. Batten, N. Clarke, C. Lambert, D.M. Causon, On the choice of wavespeeds for the HLLC Riemann solver, SIAM J. Sci. Comput. 18 (1997) 1553.
[5] M.J. Berger, J. Oliger, Adaptive mesh refinement for shock hydrodynamics, J. Comput. Phys. 53 (1984) 484.
[6] M.J. Berger, P. Collela, Local adaptive mesh refinement for shock hydrodynamics, J. Comput. Phys. 64 (1989) 82.
[7] M.J. Berger, R.J. LeVeque, An adaptive Cartesian mesh algorithm for the Euler equations in arbitrary geometries, AIAA Paper 89-1930-CP, 1989.
[8] P.C. Campbell, K.D. Devine, J.E. Flaherty, L.G. Gervasio, J.D. Teresco, Dynamic Octree Load Balancing Using Space-filling Curves, Technical Report CS-03-01, Williams College, Department of Computer Science, 2003.
[9] R. Deiterding, Parallel Adaptive Simulation of Multi-dimensional Detonation Structures, Ph.D. thesis, University at Cottbus, 2003.
[10] H. Ji, F.-S. Lien, E. Yee, A robust and efficient hybrid cut-cell/ghost-cell method with adaptive mesh refinement for moving boundaries on irregular domains, Comput. Methods Appl. Mech. Eng. 198 (2008) 432.
[11] H. Ji, F.-S. Lien, E. Yee, Numerical simulation of detonation using an adaptive Cartesian cut-cell method combined with a cell-merging technique, Comput. Fluids 39 (2010) 1041.
[12] A.M. Khokhlov, Fully threaded tree algorithms for adaptive mesh fluid dynamics simulations, J. Comput. Phys. 143 (1998) 519.
[13] J.K. Lawder, P.J.H. King, Using state diagrams for Hilbert curve mappings, Int. J. Comput. Math. 327 (2001) 78.
[14] J.K. Lawder, P.J.H. King, Querying multi-dimensional data indexed using the Hilbert space-filling curve, ACM Sigmod Record 19 (2001) 30.
[15] H. Le, P. Moin, An improvement of fractional step methods for the incompressible Navier-Stokes equations, J. Comput. Phys. 92 (1991) 369.
[16] R. Löhner, Adaptive remeshing for transient problems, Comput. Meth. Appl. Mech. Eng. 195 (1989) 75.
[17] S.V. Patankar, Numerical Heat Transfer and Fluid Flow, Hemisphere Publishing Corporation, New York, 1980.
[18] P. MacNeice, K.M. Olson, C. Mobary, R. deFainchtein, C. Packer, PARAMESH: a parallel adaptive mesh refinement community toolkit, Comput. Phys. Commun. 330 (2000) 126.
[19] E.S. Oran, J.P. Boris, Numerical Simulation of Reactive Flow, second ed., Cambridge University Press, Cambridge, 2001.
[20] J.J. Quirk, An alternative to unstructured grids for computing gas dynamic flows around arbitrary complex two-dimensional bodies, Comput. Fluids 125 (1994) 23.
[21] L.I. Sedov, Similarity and Dimensional Methods in Mechanics, Academic Press, New York, 1959.
[22] The Shared Hierarchical Academic Research Computing Network, <http://www.sharcnet.ca>.
[23] E.F. Toro, Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction, Springer, Berlin, 1997.
[24] D.N. Williams, L. Bauwens, E.S. Oran. Detailed structure and propagation of three-dimensional detonations, in: 26th Int. Symp. Combustion, Naples, Italy, April 1996.
[25] T. Xu, H. Ji, F.-S. Lien, and F. Zhang. Drag-force in supersonic dense gas-solid flow, in: 27th Int. Symp. Shock Waves, St. Petersburg, Russia, July 2009.